

One Byte, One Rank Reversal: Auditing HumanEval Pipeline Sensitivity for Base Code Models

Anonymous Author(s)

Abstract—Published benchmark tables are often reused as though model rankings were properties of checkpoints alone. We test whether the HumanEval order of five open base-model identifiers in the Qwen2.5-Coder report transfers to one fully disclosed local pipeline. Each frozen checkpoint produced one greedy completion for all 164 tasks in BF16 on a consumer GPU; EvalPlus 0.3.1 scored HumanEval and HumanEval+ in a network-isolated container. The published order failed to transfer: StarCoder2-3B scored 3/164 rather than the displayed 31.7%, reversing its order with Qwen2.5-Coder-0.5B. Kendall’s τ_b was 0.8 and the paired StarCoder2-minus-Qwen difference was -21.95 percentage points (task-resampling interval $[-28.66, -15.85]$). A stock EvalPlus control scored 2/164. In a post-hoc counterfactual, deleting only the trailing newline appended to stripped base-model prompts raised StarCoder2 to 49/164 and restored the five-model order. A completed 2×2 newline-by-stop factorial scored its fourth cell at 50/164. The result shows that an apparently innocuous prompt byte can change a base-model score by about 28 points and reverse a portable-looking rank.

I. INTRODUCTION

HumanEval measures whether generated Python functions satisfy hidden tests [1]; HumanEval+ strengthens those tests [2]. Reports frequently place several checkpoints in one table, inviting reuse of their displayed order. Yet functional correctness depends on prompts, decoding, stopping, library behavior, sanitization, and execution policy as well as weights. For base models, a boundary byte can alter whether a decoder continues a function or begins a new top-level definition.

We audit one concrete transport claim: does the HumanEval order of five base models in Qwen2.5-Coder Table 5 [3] transfer when the same model identifiers are run through a fully disclosed local stack? This is a rank-transfer study, not an exact rerun: the source report does not identify immutable checkpoint revisions or every inference detail. Our contributions are: (1) a prespecified rank endpoint with paired task-resampling analysis; (2) a complete, container-evaluated five-model replication package; and (3) a post-hoc mechanism audit showing that one trailing newline is sufficient to explain most of a 29.9-point StarCoder2 discrepancy and the observed reversal.

II. DESIGN

A. Target and decision rule

The published low-to-high order is Qwen2.5-Coder-0.5B, StarCoder2-3B, DeepSeek-Coder-1.3B, Qwen2.5-Coder-1.5B, and Qwen2.5-Coder-3B. StarCoder2’s displayed 31.7% HumanEval and 27.4% HumanEval+ values also occur in its own report [4]; we treat these appearances as one measurement, not independent replications.

TABLE I
PUBLISHED AND LOCAL GREEDY PASS@1. HE+: HUMANEval+.

Model	Pub. HE	Local HE	Pub. HE+	Local HE+
Qwen-0.5B	28.0	39/164	23.8	33/164
StarCoder2-3B	31.7	3/164	27.4	3/164
DeepSeek-1.3B	34.8	56/164	26.8	47/164
Qwen-1.5B	43.9	64/164	36.6	56/164
Qwen-3B	52.4	86/164	42.7	70/164

The primary statistic is Kendall’s τ_b between the published order and local greedy pass@1. We paired models by task and drew 10,000 percentile task-resampling replicates with seed 2026. We declared transfer only if the order was identical ($\tau_b = 1$) and no adjacent pair reversed. We declared failure if a pair reversed and its paired 95% interval excluded zero in the reversed direction; other outcomes were inconclusive. The question, endpoint, and rule were committed before aggregate correctness was inspected.

B. Frozen pipeline and safety

We resolved the five public identifiers to immutable model revisions. All used the EvalPlus base-model HumanEval prompt, forced base mode, greedy decoding, one completion per task, 512 new tokens, BF16 weights, and an NVIDIA RTX 4070. Generation used PyTorch 2.8.0+cu128 and Transformers 4.57.6. The same completion was scored on all 164 HumanEval tasks and HumanEval+ v0.1.10 with EvalPlus 0.3.1 at a pinned revision. Generated Python ran only inside a disposable Linux container with networking disabled, credentials absent, capabilities dropped, and no writable host mount beyond the result exchange directory.

Two equivalence checks guard wrapper effects. A batch-one stop criterion was compared against the optimized decode-once implementation before the full run. We also regenerated two tasks for every checkpoint through unmodified EvalPlus; raw and sanitized records were byte-identical to retained primary records.

III. PRIMARY RESULTS

Table I gives exact counts. The four non-StarCoder positions retained their relative order, but StarCoder2 fell below Qwen-0.5B. Kendall’s τ_b was 0.8 with task-resampling interval $[0.6, 0.8]$. StarCoder2 minus Qwen-0.5B was -21.95 points, interval $[-28.66, -15.85]$, so the prespecified decision was *failed to transfer*. HumanEval+ preserved the same local order. Exact percentage equality was never the endpoint.

TABLE II
STARCODER2 2×2 FACTORIAL OVER 164 TASKS. “DEF” AND “REPEAT”
CLASSIFY RAW SUFFIX TEXT; HE IS HARDENED PASS@1.

Condition	Empty	Def	Repeat	HE
Newline, stop	142	0	0	3
Newline, no stop	2	152	63	17
No newline, stop	0	0	0	49
No newline, no stop	0	143	38	50

A full StarCoder2 run through the unmodified EvalPlus generation path scored 2/164 on both suites. The optimized wrapper therefore does not explain the low score or reversal.

IV. MECHANISM AUDIT

The retained StarCoder2 suffix for HumanEval/0 began by repeating the complete function signature and docstring. EvalPlus registers the exact text `\ndef` as a generation stop for direct HumanEval completion, truncating at that boundary and leaving an empty candidate body. Across 164 stock tasks, 142 retained suffixes were empty.

EvalPlus 0.3.1 constructs base prompts as `task["prompt"].strip() + "\n"`. We reran every model while retaining all other inputs and stops but omitting that final newline. StarCoder2 rose from 3 to 49 HumanEval passes and from 3 to 42 HumanEval+ passes; 46 HumanEval tasks changed fail-to-pass and none regressed. Every other checkpoint moved by at most three net tasks. The published order was recovered ($\tau_b = 1.0$, task-resampling interval $[0.8, 1.0]$). Because this condition was selected after the primary result, it explains the local failure but does not replace the primary decision or identify the unpublished historical source configuration.

We then completed the 2×2 newline-by-`\ndef` stop factorial (Table II). Removing the newline eliminated empty suffixes. Removing the stop exposed multi-definition continuation text in both prompt conditions: without either intervention, 143 suffixes contained a top-level definition and 38 repeated the task entry point. Sanitization made the functional effect far smaller. Of 164 no-newline/no-stop candidates, 151 were byte-identical to the evaluated no-newline/standard-stop candidates;

all 13 differences were baseline failures. Hardened evaluation scored the fourth cell at 50/164 HumanEval and 43/164 HumanEval+, just one additional pass on each suite. Thus the newline accounts for nearly all functional recovery, while the stop mostly controls otherwise truncated continuation text.

V. IMPLICATIONS AND THREATS

Benchmark values should be reported as properties of complete evaluation systems. At minimum, reports should preserve exact prompt bytes, decoding, stopping, sanitizer and evaluator revisions, checkpoint revisions, and task-level outcomes. Prompt construction deserves tokenizer-boundary tests across model families rather than visual inspection of source strings.

The study transports a historical 2024 comparison to one 2026 local stack. It does not prove that the pinned revisions are the historical snapshots, identify the source pipeline, estimate hardware variation, or generalize beyond HumanEval’s 164 Python tasks. The diagnostic is post hoc and no pre-truncation logits were retained. Task-resampling intervals describe stability to task composition, not uncertainty about this fully enumerated benchmark. Broader benchmarks such as BigCodeBench [5] and LiveCodeBench [6] test different capabilities; our claim is limited to rank transport under the disclosed HumanEval pipeline.

VI. ARTIFACT AND CONCLUSION

An anonymized, history-free supplement contains model revisions, raw and sanitized generations, hardened evaluator outputs and image digests, task-level analyses, checksums, and reproduction instructions. Generated code is untrusted and must be evaluated only in the documented container.

The published five-model order did not transfer to the specified local pipeline. A one-byte prompt-boundary change raised StarCoder2 from 3/164 to 49/164 and restored the order; disabling the interacting stop added only one more pass. Portable benchmark rankings therefore require portable evaluation pipelines, not model names and aggregate percentages alone.

AI-use disclosure.: A generative AI assistant was used to draft and edit text and code; the author verified all claims, citations, computations, and artifacts.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [2] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.01210>
- [3] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [4] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, “StarCoder 2 and The Stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.19173>
- [5] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul *et al.*, “BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions,” *arXiv preprint arXiv:2406.15877*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.15877>
- [6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and contamination free evaluation of large language models for code,” *arXiv preprint arXiv:2403.07974*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.07974>